Github:-

GitHub is a web-based platform used for collaborative work and managing project-related data. It allows users to store, share, modify, and delete project data seamlessly.

Features of GitHub

- **Open Source**: Encourages open-source project development.
- **Cloud-Based**: It provides cloud storage, enabling access from anywhere.
- **Community Support**: A vast community for help and collaboration.
- **Collaboration Tools**: Tools to manage team workflows.
- **Code Reviews**: Enables peer review of code.
- **Code Hosting and Sharing**: Reliable platform to host and share code.
- **Code Management**: Tools for organizing and managing code repositories.
- **Code Security**: Security features to protect your code.
- **Code Backup**: Ensures code is safely backed up.
- **CI/CD**: Continuous Integration and Continuous Deployment support.
- **Version Control**: Ability to revert to previous versions of a project.
- **Repository**: A storage location for all project files, which tracks changes.
    - **Public**: Accessible to everyone.
    - **Private**: Accessible only to the owner or authorized users.

If There Was No GitHub

- **Local Versioning**: Managing versions locally without centralized control.
- **Email and File Sharing**: Relying on email and manual file sharing.
- **Community Isolation**: Lack of a central community hub.
- **Impact on Open Source**: Potential negative impact on open-source development.
- **Efficiency**: Reduced efficiency in collaborative work.

Github is a platform where we can store, share, track and modify data

By using github pages we can deploy the project. It is maintained by microsoft

How to Create a Repository and Upload Files Manually on GitHub

Creating a Repository

1. **Sign In to GitHub**:
   - Go to GitHub and sign in to your account.
2. **Create a New Repository**:
   - Click on the **+** icon in the upper right corner of the GitHub dashboard.
   - Select **New repository** from the dropdown menu.
3. **Repository Details**:
   - **Repository Name**: Enter a name for your repository.
   - **Description** (optional): Add a short description of your project.
   - **Public/Private**: Choose whether your repository will be public or private.
   - **Initialize Repository**:
     - Optionally, select to add a README file, .gitignore file, and a license.
4. **Create Repository**:
   - Click the **Create repository** button to finish setting up your repository.

Uploading Files Manually

1. **Navigate to Your Repository**:
   - After creating the repository, you will be directed to the repository's main page.
   - If you are returning to the repository later, navigate to it by clicking on your profile icon, selecting **Your repositories**, and clicking on the repository name.
2. **Add Files**:
   - On the repository main page, click the **Add file** button.
   - Select **Upload files** from the dropdown menu.
3. **Upload Files**:
   - Drag and drop the files or folders from your local machine into the upload area, or click **choose your files** to select files manually.
4. **Commit Changes**:
   - **Commit Message**: Enter a commit message describing the files you are uploading.
   - **Commit Options**:
     - **Commit directly to the main branch**: Choose this if you want the changes to be applied immediately.

        - **Create a new branch for this commit and start a pull request**: Choose this if you want to create a new branch and start a pull request.
    - Click the **Commit changes** button to upload the files to the repository.

How to deploy project in github-pages:-

deploy the source code into the github repository

1) Configure GitHub Pages:

- Go to your repository on GitHub.
- Click on the "Settings" tab.
- Scroll down to the "GitHub Pages" section.
- Under "Source", select the branch that contains your website files (usually main or master).
- Click "Save".

2) Access Your Website:

- Once GitHub Pages finishes deploying your site, you can access it at https://<username>.github.io (replace <username> with your GitHub username).

3) Updating Your Website

Whenever you make changes to your website files, make sure to use add, commit and push those changes to GitHub. GitHub Pages will automatically update your site after you push changes to the configured branch

.

Topic: Git intro and git config

Git (global information tracker):-

Git is a distributed version control system (VCS) that tracks changes in any set of computer files, usually used for coordinating work among programmers who are collaboratively developing source code during development.

Git can acts as a medium between development environment to storing platform

Git features Open Source and Community Support, Branching and Merging, Distributed Version Control, Staging Area by using git we can do pull request and push the code

Three components of Git

The three areas of Git refer to the three main components where Git manages and stores data internally.

Working Directory: The working directory is a directory on your system where you'll manipulate files.

Staging Area: Staging area acts as a medium between your working directory and repository.

Repository: It is referred as git respository where you will track and commit changes

Git installation

Download Git Installer: Visit the official Git website at https://git-scm.com/ and navigate to the download page.

Verify Installation: After the installation is complete, open a terminal or command prompt and run “git --version”

Verify configuration

git config --list

Setting Configuration: Set config using below commands

```
git config --global user.email "your@email.com"

git config --global user.name "Your github Name"
```

Git process 2

Process-2

In Github

1) create repository in github with your project name
2) add the description of project
3) select the security of file (public or private)
4) create the repository
5) below command lines is displayed
6) open your working directory

In your working directory

1) open the source code
2) open the terminal
3) paste the below commands one by one

git init

```
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin git@github.com:saiteja-yernagula/b-n-.git
git push -u origin main
```

git is user-friendly software so it can display the help or suggestions , if it still remains unresolved go through below websites for reference.

where to resolve issues by researching

- stack overflow
- medium
- docs
- chatgpt

Topic: Git branches

What is a Git Branch?

A Git branch is a pointer to a specific commit within the repository. It allows you to work on multiple versions of your project simultaneously. Each branch is an independent line of development.

Why Use Branches?

- **Isolation**: Branches allow you to work on new features, bug fixes, or experiments without affecting the main codebase.
- **Collaboration**: Different team members can work on different features simultaneously by creating separate branches.
- **Experimentation**: Branches are great for trying out new ideas without disrupting the main codebase.
- **Versioning**: You can use branches to maintain different versions of your project simultaneously.

Git commands for branches

1) Create a Branch: You can create a new branch with the **git branch** command:

git branch new_branch_name

2) Switch Branches: Use the **git checkout** command to switch to a different branch:

git checkout branch_name

3) Create and Switch: You can create a new branch and switch to it in one command using **-b** option:

git checkout -b new_branch_name

4) List Branches: To list all branches in your repository, you can use:

git branch

5) Merge Branches: Once you've finished work on a branch, you can merge it back into the main branch (e.g., **master**) using:

git checkout master

git merge branch_name

6) Delete Branches: After merging a branch, you can delete it with:

git branch -D branch_name

Deleting a Remote Branch:

git push origin --delete branchname

Renaming a Branch:

git branch -m old-branch-name new-branch-name

Handling Merge Conflicts: If there are conflicts during a merge, Git will pause the merge and allow you to resolve the conflicts manually. After resolving, you need to add the resolved files and complete the merge.

Best Practices for Branching

1. **Use Descriptive Names:** Choose meaningful branch names that reflect the purpose of the branch, such as feature-login, bugfix-issue123, or release-v1.2.
2. **Keep Branches Short-Lived:** Regularly merge changes back into the main branch to avoid large merge conflicts.
3. **Regularly Update Branches:** Frequently pull changes from the main branch to keep your branch up-to-date and minimize conflicts.
4. **Use Feature Branches:** Create a new branch for each new feature, bug fix, or experiment.
5. **Clean Up Stale Branches:** Regularly delete branches that are no longer needed to keep the repository clean.

Git Conflicts

What is a Git Conflict?

A Git conflict occurs when changes from different branches or contributors cannot be automatically merged.

Conflicts typically arise when two or more branches have modified the same lines of a file or when one branch has deleted a file that another branch has modified.

Common Causes of Git Conflicts

1. **Concurrent Changes**: Multiple developers making changes to the same lines in a file.
2. **Deletions and Modifications**: One branch deletes a file while another modifies it.
3. **Branching and Merging**: Frequent branching and merging without synchronization.

Identifying Conflicts

- Git automatically flags conflicts during a merge or rebase.
- Conflict markers are inserted into files to highlight the differing changes.
- Example of conflict markers:

Resolving Conflicts

1. **Manual Resolution**:
    - Open the conflicted file.
    - Decide which changes to keep, modify, or combine.

    - Remove conflict markers.
2. **Use Git Tools**:
    - Use GUI tools like GitKraken, Sourcetree, or VSCode's built-in merge conflict resolver.
    - These tools provide a visual interface to resolve conflicts.
3. **Commit Resolution**:
    - After resolving conflicts, stage the resolved files:

      git add <resolved_file>

    - Commit the resolution:

      git commit